

Spring RESTful Web Services and MockMVC – Hands-on

Common Project Configuration

application.properties

```
server.port=8083
logging.level.org.springframework=info
logging.level.com.cognizant.springlearn=debug
```

Country.java

```
package com.cognizant.springlearn;

public class Country {
    private String code;
    private String name;

    public Country() { }

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="in" class="com.cognizant.springlearn.Country">
        <property name="code" value="IN"/>
        <property name="name" value="India"/>
    </bean>
    <bean id="us" class="com.cognizant.springlearn.Country">
        <property name="code" value="US"/>
        <property name="name" value="United States"/>
    </bean>
    <bean id="jp" class="com.cognizant.springlearn.Country">
        <property name="code" value="JP"/>
        <property name="name" value="Japan"/>
    </bean>
    <bean id="de" class="com.cognizant.springlearn.Country">
        <property name="code" value="DE"/>
        <property name="name" value="Germany"/>
    </bean>
</beans>
```

```
</bean>

<bean id="countryList" class="java.util.ArrayList">
  <constructor-arg>
    <list>
      <ref bean="in"/>
      <ref bean="us"/>
      <ref bean="jp"/>
      <ref bean="de"/>
    </list>
  </constructor-arg>
</bean>
</beans>
```

Exercise 1: Understand HTTP Request and Response

Aim: To understand the main parts of an HTTP request and response before implementing REST endpoints. A request contains the method, resource path, HTTP version and headers. A response contains the HTTP version, status code, headers, content type and response body.

```
spring-learn REST - Output 1

GET /hello HTTP/1.1
Host: localhost:8083
User-Agent: PostmanRuntime/7.44.1
Accept: */*

HTTP/1.1 200 OK
Content-Type: text/plain;charset=UTF-8

Hello World!!
```

Figure 1: Output for Understand HTTP Request and Response

Result: The request and response structure was identified correctly, including method, URL, headers, status code, content type and body.

Exercise 2: Hello World RESTful Web Service

Aim: To create a GET endpoint at /hello that returns the text Hello World!! and records START and END logs.

HelloController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START");
        String message = "Hello World!!";
        LOGGER.info("END");
        return message;
    }
}
```

spring-learn REST - Output 2

```
GET http://localhost:8083/hello
Status: 200 OK
Response time: 42 ms

Hello World!!
```

Figure 2: Output for Hello World RESTful Web Service

Result: The /hello endpoint returned Hello World!! with HTTP 200.

Exercise 3: REST – Country Web Service

Aim: To return the India bean as JSON from /country. Spring converts the returned Country object into JSON automatically through the web starter.

CountryService.java

```
package com.cognizant.springlearn.service;

import java.util.List;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;

import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.exception.CountryNotFoundException;

@Service
public class CountryService {

    public Country getIndia() {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        return context.getBean("in", Country.class);
    }

    @SuppressWarnings("unchecked")
    public List<Country> getAllCountries() {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        return context.getBean("countryList", List.class);
    }

    public Country getCountry(String code) {
        return getAllCountries().stream()
            .filter(c -> c.getCode().equalsIgnoreCase(code))
            .findFirst()
            .orElseThrow(CountryNotFoundException::new);
    }
}
```

CountryController.java

```
package com.cognizant.springlearn.controller;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;

import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;

@RestController
public class CountryController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);

    private final CountryService countryService;
```

```

public CountryController(CountryService countryService) {
    this.countryService = countryService;
    LOGGER.debug("Inside CountryController Constructor");
}

@GetMapping("/country")
public Country getCountryIndia() {
    LOGGER.info("START");
    Country country = countryService.getIndia();
    LOGGER.info("END");
    return country;
}

@GetMapping("/countries")
public List<Country> getAllCountries() {
    LOGGER.info("START");
    List<Country> countries = countryService.getAllCountries();
    LOGGER.info("END");
    return countries;
}

@GetMapping({"/countries/{code}", "/country/{code}"})
public Country getCountry(@PathVariable String code) {
    LOGGER.info("START");
    Country country = countryService.getCountry(code);
    LOGGER.info("END");
    return country;
}
}

```

spring-learn REST - Output 3

```

GET http://localhost:8083/country
Status: 200 OK
Content-Type: application/json

{
  "code": "IN",
  "name": "India"
}

```

Figure 3: Output for REST – Country Web Service

Result: The /country endpoint returned the India object as JSON.

Exercise 4: REST – Get All Countries

Aim: To return all four configured countries from /countries as a JSON array.

CountryController.java

```
package com.cognizant.springlearn.controller;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;

import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;

@RestController
public class CountryController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);

    private final CountryService countryService;

    public CountryController(CountryService countryService) {
        this.countryService = countryService;
        LOGGER.debug("Inside CountryController Constructor");
    }

    @GetMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        Country country = countryService.getIndia();
        LOGGER.info("END");
        return country;
    }

    @GetMapping("/countries")
    public List<Country> getAllCountries() {
        LOGGER.info("START");
        List<Country> countries = countryService.getAllCountries();
        LOGGER.info("END");
        return countries;
    }

    @GetMapping("/{countries/{code}", "/country/{code}")
    public Country getCountry(@PathVariable String code) {
        LOGGER.info("START");
        Country country = countryService.getCountry(code);
        LOGGER.info("END");
        return country;
    }
}
```

spring-learn REST - Output 4

GET http://localhost:8083/countries

Status: 200 OK

```
[
  {"code": "IN", "name": "India"},
  {"code": "US", "name": "United States"},
  {"code": "JP", "name": "Japan"},
  {"code": "DE", "name": "Germany"}
]
```

Figure 4: Output for REST – Get All Countries

Result: The /countries endpoint returned all four countries in the configured order.

Exercise 5: REST – Get Country by Country Code

Aim: To accept a country code through a path variable and perform a case-insensitive search.

CountryController.java

```
package com.cognizant.springlearn.controller;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;

import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;

@RestController
public class CountryController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);

    private final CountryService countryService;

    public CountryController(CountryService countryService) {
        this.countryService = countryService;
        LOGGER.debug("Inside CountryController Constructor");
    }

    @GetMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        Country country = countryService.getIndia();
        LOGGER.info("END");
        return country;
    }

    @GetMapping("/countries")
    public List<Country> getAllCountries() {
        LOGGER.info("START");
        List<Country> countries = countryService.getAllCountries();
        LOGGER.info("END");
        return countries;
    }

    @GetMapping("/{countries/{code}", "/country/{code}")
    public Country getCountry(@PathVariable String code) {
        LOGGER.info("START");
        Country country = countryService.getCountry(code);
        LOGGER.info("END");
        return country;
    }
}
```

spring-learn REST - Output 5

GET http://localhost:8083/countries/in

Status: 200 OK

```
{
  "code": "IN",
  "name": "India"
}
```

Figure 5: Output for REST – Get Country by Country Code

Result: The endpoint returned India even when the path code was supplied in lowercase.

Exercise 6: REST – Exceptional Scenario

Aim: To return a meaningful 404 response when the requested country code is not available.

CountryNotFoundException.java

```
package com.cognizant.springlearn.service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(value = HttpStatus.NOT_FOUND,
                reason = "Country not found")
public class CountryNotFoundException extends RuntimeException {
    private static final long serialVersionUID = 1L;
}
```

spring-learn REST - Output 6

```
GET http://localhost:8083/country/az
Status: 404 Not Found

{
  "status": 404,
  "error": "Not Found",
  "message": "Country not found",
  "path": "/country/az"
}
```

Figure 6: Output for REST – Exceptional Scenario

Result: An invalid code produced HTTP 404 with the message Country not found.

Exercise 7: MockMVC – Test Get Country Service

Aim: To verify that CountryController loads correctly and that /country returns status 200 with code IN and name India.

SpringLearnApplicationTests.java

```
package com.cognizant.springlearn;

import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.web.servlet.MockMvc;

import com.cognizant.springlearn.controller.CountryController;

@SpringBootTest
@AutoConfigureMockMvc
class SpringLearnApplicationTests {

    @Autowired
    private CountryController countryController;

    @Autowired
    private MockMvc mvc;

    @Test
    void contextLoads() {
        assertNotNull(countryController);
    }

    @Test
    void testGetCountry() throws Exception {
        mvc.perform(get("/country"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").exists())
            .andExpect(jsonPath("$.code").value("IN"))
            .andExpect(jsonPath("$.name").exists())
            .andExpect(jsonPath("$.name").value("India"));
    }

    @Test
    void testGetCountryException() throws Exception {
        mvc.perform(get("/country/az"))
            .andExpect(status().isNotFound())
            .andExpect(status().reason("Country not found"));
    }
}
```

spring-learn REST - Output 7

```
C:\spring-learn> mvn clean test
[INFO] Running com.cognizant.springlearn.SpringLearnApplicationTests
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Figure 7: Output for MockMVC – Test Get Country Service

Result: All controller and response assertions passed successfully.

Exercise 8: MockMVC – Test Exceptional Scenario

Aim: To verify the error path through MockMVC when an unknown country code is requested.

The exceptional test is included in `SpringLearnApplicationTests.java` shown in the previous exercise. It calls `/country/az` and verifies the 404 status and reason.

```
spring-learn REST - Output 8

MockMvc GET /country/az
Resolved Exception: CountryNotFoundException
MockHttpServletRequest:
Status = 404
Error message = Country not found

Tests run: 3, Failures: 0, Errors: 0
```

Figure 8: Output for MockMVC – Test Exceptional Scenario

Result: The exceptional MockMVC test passed with the expected 404 response.